MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Old Bugs, New Protocol: A Root-Cause Analysis of Security Failures in the Model Context Protocol Ecosystem Under Rapid Adoption

Erori vechi, protocol nou: analiza cauzelor eșecurilor de securitate din ecosistemul Model Context Protocol în condițiile adoptării accelerate

Research article

Master programme: Software Engineering

Student: Negai Marin, group IS-261M

Supervisor: _____

Chișinău, 2026

Abstract

Negai Marin, *"Old Bugs, New Protocol: A Root-Cause Analysis of Security Failures in the Model Context Protocol Ecosystem Under Rapid Adoption"*, research article, Technical University of Moldova, Faculty of Computers, Informatics and Microelectronics, Chișinău, 2026. The article has an introduction, four chapters, conclusions, a bibliography of 30 sources and one appendix. It contains 5 figures and 5 tables.

Keywords: Model Context Protocol, AI agents, software security, prompt injection, vulnerability analysis.

The Model Context Protocol (MCP) was released at the end of 2024 as a common way to connect language-model assistants to files, databases and online services. Within a year it was supported by most large AI vendors, and during the same period security researchers published dozens of reports about vulnerable servers, compromised packages and attacks that hide instructions inside tool descriptions. These reports are usually discussed as a new "AI security" problem. The question behind this article is whether that label is accurate, or whether most of the failures are ordinary software defects that appeared because the ecosystem grew faster than anyone could review it.

The objective was to find where MCP security failures come from. I collected 54 publicly documented weaknesses and incidents disclosed between March 2025 and February 2026, mainly from an open vulnerability catalogue and partly from the original research disclosures. Every record was read in its original source and coded on two axes. The first axis is the nature of the flaw: classic (it would exist even without a language model), AI-native (it works only because a model follows text it should not trust) or hybrid. The second axis is the stage where the flaw was introduced: protocol design, implementation, integration design, supply chain or deployment. All counts and charts were produced by a small Python script from the coded dataset, which is published together with the article.

The results show that 61.1% of the records are classic weaknesses such as command injection, path traversal, server-side request forgery and missing authentication, and half of all records are coding mistakes in individual servers, clients or tools. AI-native weaknesses make up 29.6% of the sample and come almost entirely from the design of the protocol and of the integrations built on it. None of them received a CVE identifier, which means that tools based on vulnerability databases cannot see this part of the risk. The article ends with a set of controls for developers, operators and public institutions, and compares the findings with the October 2026 breach of Moldovan e-government services, where a comparable trust error led to the exposure of personal data of about 450,000 citizens.

Adnotare

Negai Marin, *"Erori vechi, protocol nou: analiza cauzelor eșecurilor de securitate din ecosistemul Model Context Protocol în condițiile adoptării accelerate"*, articol științific, Universitatea Tehnică a Moldovei, Facultatea Calculatoare, Informatică și Microelectronică, Chișinău, 2026. Articolul este alcătuit din introducere, patru capitole, concluzii, o bibliografie din 30 de surse și o anexă. Lucrarea conține 5 figuri și 5 tabele.

Cuvinte-cheie: Model Context Protocol, agenți AI, securitate software, injectare de prompt, analiza vulnerabilităților.

Model Context Protocol (MCP) a fost lansat la sfârșitul anului 2024 ca o modalitate comună de a conecta asistenții bazați pe modele de limbaj la fișiere, baze de date și servicii online. În decurs de un an, protocolul a fost preluat de majoritatea companiilor mari din domeniul AI, iar cercetătorii au publicat zeci de rapoarte despre servere vulnerabile și atacuri care ascund instrucțiuni în descrierile instrumentelor. Aceste probleme sunt prezentate de obicei ca un risc nou, specific AI. Articolul verifică dacă această interpretare este corectă sau dacă majoritatea eșecurilor sunt defecte software obișnuite, apărute pentru că ecosistemul a crescut mai repede decât a putut fi verificat.

Obiectivul lucrării a fost identificarea surselor eșecurilor de securitate din MCP. Au fost colectate 54 de vulnerabilități și incidente documentate public, dezvăluite între martie 2025 și februarie 2026, dintr-un catalog deschis de vulnerabilități și din publicațiile originale. Fiecare înregistrare a fost citită în sursa originală și clasificată pe două axe. Prima axă descrie natura defectului: clasic (ar exista și fără un model de limbaj), specific AI (funcționează doar pentru că modelul urmează un text în care nu ar trebui să aibă încredere) sau hibrid. A doua axă indică etapa în care a apărut defectul: proiectarea protocolului, implementarea, proiectarea integrării, lanțul de aprovizionare sau configurarea la instalare. Toate valorile au fost calculate de un script Python pe baza setului de date publicat.

Rezultatele arată că 61,1% dintre înregistrări sunt vulnerabilități clasice, precum injectarea de comenzi, traversarea căilor, falsificarea cererilor din partea serverului și lipsa autentificării, iar jumătate din totalul înregistrărilor sunt greșeli de programare în servere, clienți sau instrumente. Vulnerabilitățile specifice AI reprezintă 29,6% din eșantion și provin aproape în totalitate din proiectarea protocolului și a integrărilor. Niciuna dintre ele nu a primit un identificator CVE, deci instrumentele bazate pe baze de date de vulnerabilități nu pot observa această parte a riscului. Articolul propune un set de măsuri pentru dezvoltatori, operatori și instituții publice și compară rezultatele cu incidentul din octombrie 2026 care a afectat serviciile de guvernare electronică din Republica Moldova, unde o eroare de încredere asemănătoare a dus la expunerea datelor personale a aproximativ 450.000 de cetățeni.

Content

List of abbreviations **5**

Introduction **6**

1 Background and Related Work **7**

1.1 How the Model Context Protocol works 7

1.2 A protocol that grew faster than its security model 8

1.3 Related work 8

2 Methodology **9**

2.1 Data sources and selection 10

2.2 Classification scheme 10

2.3 Procedure and reliability 11

3 Results **11**

3.1 Classic versus AI-native weaknesses (RQ1) 11

3.2 Where the weaknesses originate (RQ2) 12

3.3 A blind spot in vulnerability tracking 13

3.4 Signs of rushed engineering (RQ3) 13

4 Discussion **14**

4.1 Why rapid adoption produced old bugs 14

4.2 What is specific to AI 15

4.3 A local parallel: the 2026 Moldovan e-government breach 15

4.4 Recommended controls 16

4.5 Limitations 16

Conclusions **17**

Bibliography **18**

Appendix A. Analysed records **20**

Appendix B. AI authorship check **22**

List of abbreviations

API - Application Programming Interface. A defined way for one program to request data or actions from another, usually over HTTP

CVE - Common Vulnerabilities and Exposures. A public catalogue that gives each confirmed security flaw in a specific product a unique identifier, for example CVE-2025-6514

CVSS - Common Vulnerability Scoring System. A 0 to 10 scale used to rate the severity of a vulnerability; scores of 9.0 and above are considered critical

IDNP - Personal identification number (Numărul de identificare de stat). The 13-digit number assigned to every citizen of the Republic of Moldova and used as the key in state registries

LLM - Large Language Model. A neural network trained on large amounts of text that generates answers and, in agent systems, decides which tools to call

MCP - Model Context Protocol. An open standard, published in 2024, that defines how AI applications discover and call external tools and data sources

RCE - Remote Code Execution. A vulnerability that lets an attacker run their own commands on another machine, usually the most severe outcome of an attack

SDK - Software Development Kit. A library provided by the protocol maintainers that developers use to build servers and clients

SSRF - Server-Side Request Forgery. A flaw that makes a server send requests to addresses chosen by the attacker, often internal services or cloud metadata endpoints

VMP - Vulnerable MCP Project. An openly maintained online catalogue of documented MCP weaknesses, used as the main data source of this study

Introduction

Over the last two years, AI assistants have moved from answering questions to performing tasks. A coding assistant now opens files, runs tests and creates pull requests; an office assistant reads the mailbox and books meetings. To do this, the model needs a way to reach external systems. Until late 2024 every vendor built its own connectors. In November 2024 Anthropic published the Model Context Protocol (MCP), an open specification that describes how an AI application finds the tools offered by an external program and how it calls them [1]. The protocol was adopted quickly. OpenAI and Google announced support in the spring of 2025, and in December 2025 the project was transferred to the Agentic AI Foundation, which operates under the Linux Foundation [2]. In July 2026 Netskope reported that MCP traffic among its customers had grown four times in only ten weeks [3].

The security of the ecosystem developed more slowly. The first version of the specification did not include any authorisation mechanism. Critical vulnerabilities were found in official reference servers and in the developer tools published by the maintainers, and internet-wide scans in 2026 found thousands of servers that accepted requests without checking who sent them [4, 5]. In May 2026 the U.S. National Security Agency released a separate information sheet on the secure use of MCP [6].

In public discussion these problems are usually grouped under the heading of "AI security". For part of them the label fits. Some attacks, such as hiding instructions inside the description of a tool, only work because a language model reads that text and acts on it. Many other reports, however, describe flaws that have been known for decades: a server that inserts user input into a shell command, a file tool that can be tricked into leaving its directory, a debugging service that listens on all network interfaces without a password. While reading the first advisories for this topic, I had the impression that the second kind was more common than the discussion suggested. If this impression is right, then the defences already exist and are well documented, and the real question is why they were skipped while the ecosystem was growing.

Motivation and relevance. For a software engineer the topic is practical. MCP servers are written by ordinary developers, very often as small open-source projects, and they carry the habits and shortcuts of any code written under time pressure. Meanwhile, companies and public institutions are planning to connect AI assistants to their internal systems. In Moldova this question became concrete in October 2026, when the e-government infrastructure suffered a data breach caused by an error in how access rights were checked [7]. The MCP security guidance warns about a related class of errors, in which an intermediary is trusted in place of the party that actually makes the request [8].

Objective and research questions. The objective of the article is to establish where MCP security failures originate. The work is organised around three questions:

- **RQ1:** What share of the documented MCP weaknesses are classic software flaws, and what share depend on the behaviour of a language model?
- **RQ2:** At which stage (protocol design, implementation, integration, supply chain or deployment) were these weaknesses introduced?
- **RQ3:** Which signs of rushed engineering can be seen in the records, and what should teams that adopt MCP do about them?

Methodology. The study is a structured review of secondary data. I collected 54 documented weaknesses and incidents from a public vulnerability catalogue and from the original research reports, classified each of them using fixed rules, and compared the results with large measurement studies published in 2026. No real systems were scanned or tested.

1 Background and Related Work

1.1 How the Model Context Protocol works

MCP uses a client–server architecture with three roles [9]. The *host* is the application the user works with, such as a desktop chat assistant or a code editor with AI features. For every external program it connects to, the host creates a separate *client*. That external program is called a *server*, and it offers three kinds of capabilities. *Tools* are functions the model is allowed to call, for example "create issue" or "run query". *Resources* give the model read-only context, such as the content of a file. *Prompts* are prepared templates for common tasks. Client and server exchange JSON-RPC messages. A local server is started by the host as a child process and communicates through standard input and output (stdio); a remote server is reached over HTTP.

From a security point of view, the most important part is the way tools are offered to the model. After connecting, the server sends a list of its tools. Each entry has a name, a short description written in natural language and a schema of the parameters. The host adds these descriptions to the model's context, and the model then decides by itself which tool to call and when. This means that text written by whoever published the server reaches the model almost like a request from the user. The results returned by tools, which can include the content of web pages, tickets or emails, end up in that context too. Figure 1.1.1 marks the trust boundaries of this design and the four places where text controlled by an attacker can enter the system.

Entry points 1 and 2 are dangerous only because a language model interprets the text that passes through them. Entry points 3 and 4 are not specific to AI at all: any software package can be replaced by a malicious copy, and any network service can be reached by someone who should not have access. The classification used in this study is built on this division.

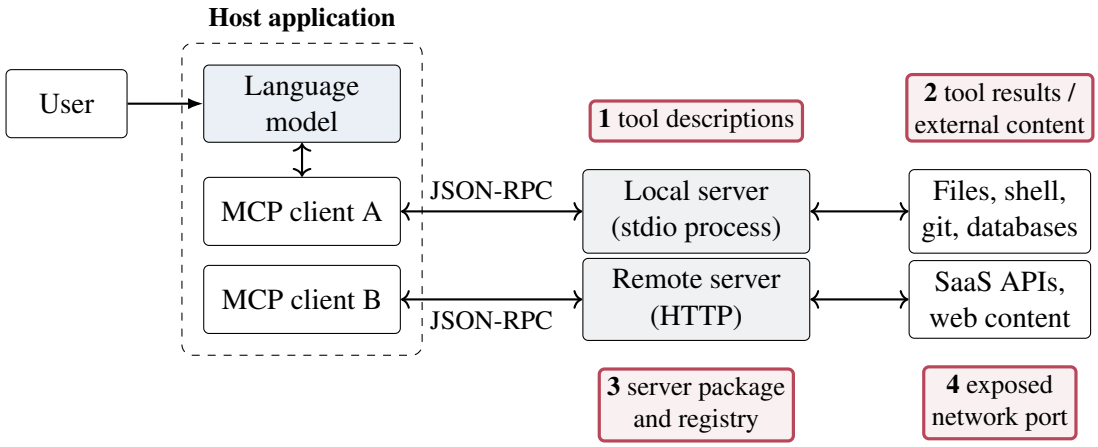


Figure 1.1.1 - Simplified MCP architecture and the four places where attacker-controlled input enters (author's diagram based on [9, 8])

1.2 A protocol that grew faster than its security model

The specification has gone through several revisions, and a number of security features were added only after the related problems had already been reported. Figure 1.2.1 shows the main revisions next to some of the best-known disclosures. The version from November 2024 had no authorisation scheme. OAuth-based authorisation was introduced in the revision of March 2025. In June 2025 the model was changed again: an MCP server now acts only as an OAuth resource server, and clients have to bind each token to the server it is meant for. The accompanying security guidance explicitly prohibits "token passthrough", a practice in which a server takes a token it received and forwards it to another API without checking that the token was issued for itself [8]. In July 2026 the protocol dropped its own session mechanism, and with it the problems caused by session identifiers [9]. A young standard is expected to change often. In this case, though, each revision arrived when many servers were already running in production.

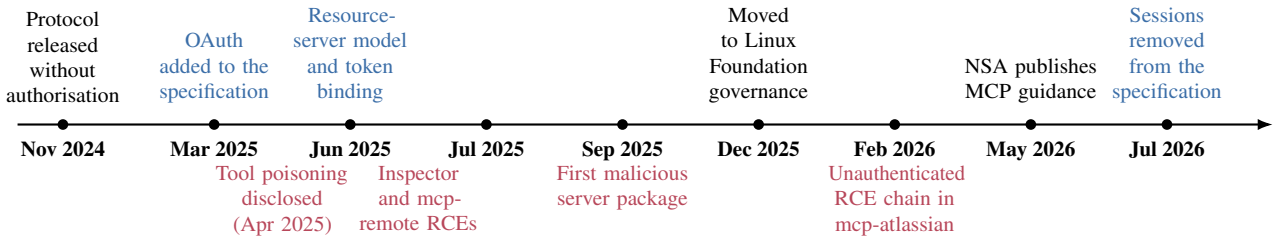


Figure 1.2.1 - Specification revisions (blue) and governance milestones (black) against selected security events (red) (compiled by the author from [1, 9, 8, 2, 6, 10, 11, 12, 13, 14])

1.3 Related work

Published research on MCP security can be divided into three groups. Studies in the first group build threat models. Hou et al. followed the life cycle of an MCP server through four phases (creation, deployment, operation and maintenance) and described 16 threat scenarios, which they grouped by four types of attackers; installer spoofing and tool name conflicts are two of these scenarios [15].

Radosevich and Halloran connected popular models to standard servers and showed that the models could be led into running malicious code, opening remote access and leaking credentials [16]. These studies explain which attacks are possible, but they do not say how often each type occurs in practice.

The second group focuses on attacks that go through the model. The general technique, indirect prompt injection, was described by Greshake et al., who planted instructions in content that an application later retrieved and showed that this was enough to take control of an LLM-based system [17]. Prompt injection is also listed first in the OWASP Top 10 for LLM applications [18]. In the MCP context, Invariant Labs used the term *tool poisoning* for instructions hidden in tool descriptions [10], and in May 2025 demonstrated that a single public GitHub issue could make an agent copy data from private repositories [19]. The MCPTox benchmark measured the problem on a larger scale. With 1,348 poisoned tool definitions based on 45 real servers, the most affected of 20 tested models followed the hidden instructions in 72.8% of cases, and even the most careful one refused fewer than 3% [20].

The third group looks at real deployments, and its results are collected in Table 1.3.1. Two of their findings matter most for this article. Servers without proper authentication are common on the public internet, and many servers exist only for a few days, which makes it hard to keep track of them or patch them. He et al. took a different route and built a cleaned dataset of CVE records related to MCP [21].

Table 1.3.1 - Deployment-level evidence from 2026 measurement studies

Study	Sample	Key finding
Zhou et al. [4]	7,973 live remote servers	40.55% expose tools without any authentication; 29% use static keys; all 119 OAuth servers tested had at least one flaw
Padilla [5]	414 audited servers	91.8% lack OAuth; 41.6% of servers seen in one scan disappeared within three days
He et al. [21]	24 validated CVEs	87.5% rated Critical or High; 75% caused by insufficient input validation

Research gap. Every group of studies covers only one part of the problem. Work based on CVE records sees mainly implementation bugs, because weaknesses that depend on a language model rarely get an identifier. Work on prompt injection sees the model-level risk but not the ordinary bugs around it. I did not find a study that puts both kinds of weaknesses into the same dataset and asks where each one was introduced. This article attempts that on a small but fully documented sample.

2 Methodology

The study is based on documents rather than experiments: each documented weakness is one unit of analysis, classified by fixed rules and then counted. Figure 2.0.1 shows the order of the steps. The dataset and the script that calculates every number in Chapter 3 are kept in the repository together

with the source of this article, so anyone can repeat the analysis or classify the records again.

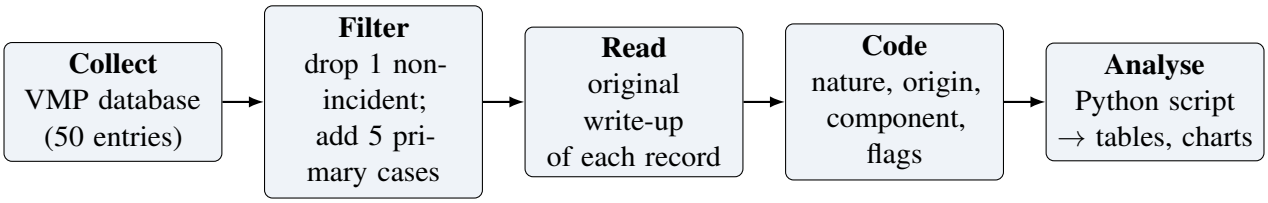


Figure 2.0.1 - Research workflow used to build and analyse the dataset

2.1 Data sources and selection

The main source is the Vulnerable MCP Project (VMP), an open online catalogue of MCP weaknesses in which every entry links to the original report [22]. I preferred it to the CVE list because it also includes weaknesses that never received an identifier, for example attack techniques that target the protocol itself. When the data was collected, on 10 November 2026, the catalogue had 50 entries. One of them describes an academic paper and not a specific weakness, so it was excluded, which left 49 records.

During the literature review I came across several well-documented incidents that were missing from the catalogue. Relying on a single curator is a source of bias, so I added five of these cases, each taken from its original disclosure: the sandbox escape in the official filesystem server [23], the malicious `postmark-mcp` package [13], a path traversal in a platform that hosts MCP servers [24], the exposure of data between customers of Asana's MCP integration [25], and the unauthenticated attack chain in `mcp-atlassian` [14]. The final dataset has 54 records disclosed between March 2025 and February 2026.

A record was included if the weakness was publicly documented in enough technical detail to identify its root cause and if it concerned an MCP component or a behaviour reached through MCP. Several CVE identifiers published together for one component, such as the three issues in the git server, were counted as one record, since they shared a cause and a fix.

2.2 Classification scheme

Every record received a value on two main axes and three additional attributes, all listed in Table 2.2.1. The hardest decision was the *nature* of a weakness, so I used one control question to keep it consistent: *would this weakness still exist if the language model were replaced by ordinary code that calls the same tools?* If the answer is yes, the weakness is *classic*; if the attack only works by convincing a model through text, it is *AI-native*. The *hybrid* category covers two situations: a classic flaw that can only be reached through prompt injection, and an old type of flaw that appears only because of the way agents behave.

Two borderline cases, records R25 and R21 in Appendix A, illustrate how the rules were

Table 2.2.1 - Coding scheme applied to every record

Axis	Value	Decision rule
Nature	Classic	A known weakness class (e.g. injection, traversal, SSRF, missing authentication) that is exploitable without any model in the loop.
	Hybrid	A classic flaw that is reached through prompt injection, or a classic class of flaw that only appears because of agent behaviour.
	AI-native	The attack depends on a model treating untrusted text as instructions; no code fix of a single component removes it.
Origin	Protocol design	Caused by what the specification allows or does not require.
	Implementation	A coding error in a server, client, host, SDK or tool.
	Integration design	Each part works as built, but the way they are combined (privileges, data access, missing confirmation) creates the risk.
	Supply chain	Introduced through distribution: packages, registries, hosting.
	Deployment	Caused by default configuration or operator setup.
Component	Server, Host/client, SDK, Developer tooling, Protocol, Ecosystem	
Flags	CVE assigned; insecure default involved; first-party (official project) code	

applied. The weakness known as "MCPoison" in the Cursor editor allowed an attacker to change a project's MCP configuration after the user had already approved it [22]. The editor never checked the file again, which is a missing integrity check with no model involved, so the record is classic, with implementation as its origin. In the zero-click attack through a shared document, the model reads hidden instructions and then runs a command. The result is ordinary code execution, but it cannot be reached without the model, so this record is hybrid, originating in integration design.

2.3 Procedure and reliability

For every record I opened the advisory or research post linked from the catalogue, summarised the root cause in one sentence and only then assigned the values. All records were classified by one person, a limitation discussed in Section 4.5. To reduce its effect, the rules were fixed before classification began, and every value is stored in a CSV file next to its source.

3 Results

3.1 Classic versus AI-native weaknesses (RQ1)

Out of 54 records, 33 (61.1%) were classified as classic, 16 (29.6%) as AI-native and 5 (9.3%) as hybrid. The catalogue intentionally collects attack techniques against the protocol, and still about three in five of the records are problems that are covered in a standard web security course. Four families account for 25 of the 33 classic records. Command injection (7 records) appears, for example, in the Kubernetes, Figma and GitHub Kanban servers, and path traversal (6) in the filesystem, git and Zen servers. Servers that download content from URLs suffer from Server-Side Request Forgery (SSRF) (3). The largest family, with 9 records, is missing access control: services that accept requests without authentication or authorisation, or that any web page can reach through DNS rebinding. What

is left are malicious packages, data leaking between clients or tenants, missing integrity checks and exposed credentials. Some of the classic flaws allow Remote Code Execution (RCE) and have very high severity scores: 9.6 on the CVSS scale for the command injection in `mcp-remote` [11], 9.4 for the unauthenticated proxy in the official MCP Inspector [12], and 9.1 for the file-write flaw in `mcp-atlassian`, a package downloaded more than four million times [14].

The AI-native records are fewer and much more similar to each other. Nearly all of them rely on text that the user never sees but that the model reads and follows. In 10 of them the text is supplied by a connected server. This covers tool poisoning, "rug pulls" (a server changes the description of a tool after the user has approved it) and tool shadowing, where one server influences how the tools of another server are used. In the other 6 the text arrives inside the data that tools return, for example a GitHub issue or a Jira ticket.

Figure 3.1.2 adds the time of disclosure. Most AI-native records date from the second quarter of 2025, when tool poisoning, rug pulls and shadowing were described by different researchers within a few weeks. From the third quarter on, the majority of new records are classic implementation bugs in third-party servers. My interpretation is that design-level attacks follow directly from how the protocol works and were noticed early, while implementation bugs accumulated as more servers were written. The last bar covers only January and February 2026.

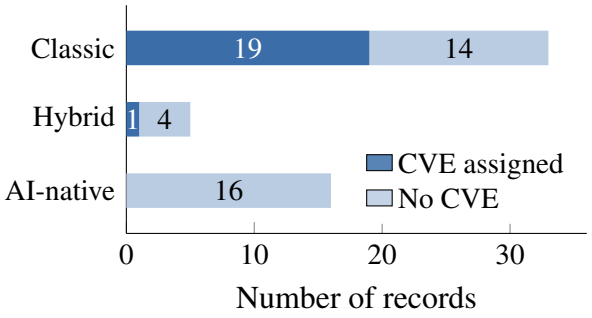


Figure 3.1.1 - Records by nature and CVE assignment (n = 54)

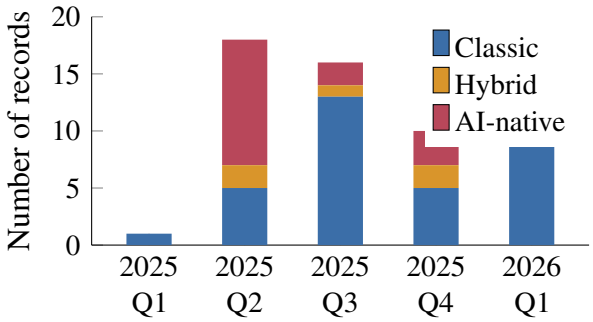


Figure 3.1.2 - Records by quarter of disclosure and nature

3.2 Where the weaknesses originate (RQ2)

Table 3.2.1 combines the nature of each record with the stage at which the weakness was introduced, and the two axes turn out to be closely linked. Classic weaknesses come almost only from implementation: 27 records, or 50.0% of the whole dataset, are ordinary programming errors in servers, clients, SDKs or developer tools. AI-native weaknesses come from design. The protocol itself permits 11 of them. The main reason is that any connected server can put arbitrary text in front of the model, and the specification does not require tool definitions to be fixed or approved again after a change. Another 4 come from integration design. In those cases each component worked as it was built, but

the agent had a token with broad permissions and was also reading untrusted content, which is what happened in the GitHub case [19].

Table 3.2.1 - Records by origin and nature

Origin	Classic	Hybrid	AI-native	Total
Implementation	27	2	0	29
Protocol design	1	1	11	13
Integration design	0	1	4	5
Supply chain	3	1	1	5
Deployment	2	0	0	2
Total	33	5	16	54

Table 3.2.2 - Records by affected component

Component	Classic	Hybrid	AI-native	Total	With CVE
Server	20	0	0	20	12
Host	4	3	5	12	3
Protocol	1	1	11	13	0
Ecosystem	3	1	0	4	0
SDK	2	0	0	2	2
Dev tooling	3	0	0	3	3

Table 3.2.2 breaks the records down by component. All 20 records about MCP servers are classic, and 12 of them have a CVE identifier. Hosts, that is editors and agent applications, have the most mixed profile. The host is where the model, the user interface and the tool connections meet.

3.3 A blind spot in vulnerability tracking

Figure 3.1.1 also shows a result that I had not planned to look for. 20 records (37.0%) have a CVE identifier, 19 of them classic. None of the AI-native records has one. The reason lies in how the CVE system works: it describes a defect in a specific version of a product, which the vendor can fix in a later version. Tool poisoning or an injection hidden in a ticket does not fit this definition, because the vulnerable "product" is a combination of a model, a protocol and a set of permissions. In practice, an organisation that checks its MCP exposure with a standard dependency scanner will only see the classic part of the risk. This also explains why the CVE-based study by He et al. found mostly injection and traversal flaws [21]; the other type of weakness does not reach that data source.

3.4 Signs of rushed engineering (RQ3)

Two attributes in the dataset are related to time pressure rather than to unknown threats. The first one is *insecure defaults*. In 9 records, or 27.3% of the classic ones, the weakness was caused by a default setting or became exploitable because of it. Typical examples are servers that listen on all network interfaces, the missing DNS rebinding protection in the official Software Development Kit (SDK)s and a debugging proxy that did not require a password. Each of these is a small configuration choice that is convenient during development and was then left in the released version.

The second attribute is *first-party code*. 8 records concern code or design published by Anthropic or by the protocol project. They cover the official SDKs, the MCP Inspector, the git and filesystem reference servers and the session design of the early specification. Two less obvious cases belong here as well: credentials that several reference servers stored in plain text, and terminal escape sequences that could hide instructions from a user of Claude Code. 7 of these records are classic.

Reference implementations are used as examples by many developers, so an error there tends to be repeated in other projects. Both observations agree with the measurement studies in Table 1.3.1, where these problems appear on a much larger scale.

4 Discussion

4.1 Why rapid adoption produced old bugs

In most of the analysed cases the security problem was an ordinary software engineering mistake that happened to be made in an AI product. Going through the original reports, I noticed five mechanisms that repeat and that connect the speed of adoption with these mistakes.

Thin wrappers around command-line tools. Many servers only pass requests on to existing programs such as `git`, `kubectl` or `curl`. The fastest way to write such a server is to build a command string from the tool arguments and hand it to the shell, and the command injection flaws in the Kubernetes, Figma and GitHub Kanban servers were created in this way. The language model plays no role in them; any client able to send a crafted argument could exploit these servers.

Local assumptions on the network. In the beginning, MCP servers ran locally over stdio, and the only program talking to them was the user's own application. When these servers were moved to HTTP, many of them kept the assumptions of the local setup: no authentication, listening on every interface and no validation of the `Host` or `Origin` headers. In the MCP Inspector case, these settings combined with browser behaviour so that opening a malicious web page was enough to execute code on the developer's computer [12].

Security added to the specification afterwards. Authorisation, token binding and the removal of sessions all came after the protocol was already in use (Figure 1.2.1). Servers written for an earlier revision are not updated automatically, so every late change leaves behind older implementations with weaker protection. This agrees with the measurement by Zhou et al., where four out of ten live servers still had no authentication in 2026 [4].

Reference code copied as a template. The official filesystem server checked whether a path was inside the allowed directory by comparing the beginning of two strings [23]. Developers who use a reference server as a starting point are likely to copy such a check, so a mistake in first-party code has a larger effect than a single record suggests.

Installation without review. Servers are installed from public registries and package managers with one command, often by people who would be much more careful with an unknown browser extension. The `postmark-mcp` package worked correctly for fifteen versions; the sixteenth added a single line that sent a hidden copy of every email to the attacker [13]. In another case, a path traversal in a hosting platform exposed a token that controlled more than three thousand hosted servers [24].

4.2 What is specific to AI

The AI-native records are a minority, but they are more difficult to deal with. Their cause is a property of current language models and not an error in one component: a model cannot reliably separate the data it is supposed to process from the instructions it is supposed to follow. Willison describes the most dangerous situation as a "lethal trifecta", in which one agent has access to private data, reads untrusted content and is able to send information outside [26]. The GitHub incident matches this description. The GitHub server worked as designed, so no patch to it would have stopped the attack [19]. The MCPTox results support this view: the more capable models were often easier to manipulate, because they are better at following instructions [20].

Protection against these weaknesses therefore has to be built into the architecture of the system; the operator controls in Table 4.4.1 are of this kind. And since CVE feeds do not contain them (Figure 3.1.1), they have to be covered during threat modelling and cannot be left to vulnerability management alone.

4.3 A local parallel: the 2026 Moldovan e-government breach

The breach of the Moldovan e-government systems in October 2026 did not involve MCP. It is discussed here because it shows what a trust error of this kind costs even without AI. According to public reports, between 19 and 24 October attackers sent automated requests from more than 638 IP addresses, with about one million requests on the busiest day, and obtained personal data linked to national identification numbers (IDNP) [27, 7, 28]. The number of affected people, first estimated at 750,000, was later corrected to about 450,000. After a meeting of the National Security Council, President Maia Sandu explained that the cause was structural. The systems verified which application was asking for data, but not who the person behind the request was or whether that person had the right to see the data. As a result, one application with incorrectly configured access opened the registries of at least ten institutions [7].

This is a form of the confused deputy problem, which Hardy described in 1988: a program with wide authority uses it on behalf of a caller who should not have it [29]. In API security the closest category is broken object-level authorisation, the first item of the OWASP API Security Top 10 [30]. The MCP guidance deals with related but narrower cases: its own confused deputy section is about an OAuth proxy, and its rule against token passthrough obliges a server to accept only tokens that were issued for it [8]. Neither is the mechanism that failed in Moldova. The principle is shared, though. Authority has to be checked for the real requester, where the data is kept, and cannot be inherited from an intermediary. For public institutions this has a practical consequence. Suppose an AI assistant is connected to state registries with this trust model, in which the assistant is trusted and the citizen's

rights are not checked. An attacker would then not need to write a script at all. Instructions hidden in a document or in a form field could be enough to make the assistant run those queries for them.

4.4 Recommended controls

Table 4.4.1 lists the controls that follow from the analysis. Most of them are well known from general secure development practice. The last column names the group of records, or the part of the Moldovan case, to which each control applies. Such a control lowers the risk for its group; whether it would have stopped every single case was not tested.

Table 4.4.1 - Recommended controls and the records they address (number of records in brackets)

Audience	Control	Addresses
Developers	Never build shell strings; pass arguments as arrays and validate them against an allow-list	Command injection (7)
	Resolve symbolic links and canonical paths before any sandbox check	Path traversal (6)
	Bind local servers to the loopback address, require a token, check `Host/Origin`	Missing access control (9)
	Block private address ranges for any URL fetched on behalf of a model	SSRF (3)
Operators	Keep an inventory of servers; pin versions and hash tool descriptions to detect silent changes	Malicious packages (2), text supplied by servers (10)
	Issue a separate, narrowly scoped token per user and per server; never reuse one broad token	Integration design (5)
	Do not combine private data, untrusted content and outbound actions in one agent session	Injected content (6)
	Require human confirmation for irreversible or external actions	AI-native (16), hybrid (5)
Public sector	Authorise every request by end-user identity at the data source, not by calling application	Moldovan case: confused deputy
	Rate-limit and monitor bulk queries on personal identifiers	Moldovan case: bulk enumeration
	Include AI integrations in security review before procurement and launch	All records

4.5 Limitations

The sample is small. With 54 records, the 95% Wilson interval for the share of classic weaknesses runs from 47.8% to 73.0%, so what the data supports is that classic weaknesses make up about half of the records or more, not an exact percentage. The data is also not current. The catalogue had not been updated between February 2026 and November 2026, when the data was collected, so weaknesses disclosed in those nine months, including any connected with the July 2026 revision of the specification, are not covered. The dataset depends largely on the choices of one public catalogue. It is likely to over-represent the first wave of design-level attacks, which researchers documented in

detail, and to under-represent ordinary bugs that were fixed without a public report; if so, the real share of classic weaknesses is higher than measured here. All records were classified by one person. The rules and data are published, but agreement between independent coders was not measured. The unit of analysis is a record, not a single vulnerability, so a chain of three related CVEs counts once, and so does a general attack technique. Some sources are blogs of security companies, which benefit from presenting problems as serious; where possible, numbers were taken from peer-reviewed papers or preprints instead. Finally, the information about the Moldovan breach comes from official statements that were still being updated while this article was written.

Conclusions

This article analysed 54 documented security weaknesses in the Model Context Protocol ecosystem to find out whether its widely discussed problems come from artificial intelligence or from the speed at which the ecosystem was built. Most of them belong to the second category.

For **RQ1**, 61.1% of the records are classic weaknesses that do not need a language model to be exploited, 29.6% are AI-native and the remaining 9.3% are hybrid. For **RQ2**, the two main groups have different origins. Classic weaknesses come mostly from implementation errors, which account for half of the dataset, while AI-native weaknesses come from the design of the protocol and of the integrations built on top of it. For **RQ3**, the clearest signs of rushed work are insecure default settings, present in more than a quarter of the classic records, errors in first-party reference code that other developers reuse, security features added to the specification after it was already in use, and the installation of servers without any review. None of the AI-native records received a CVE identifier, so tools and processes that rely on vulnerability databases see only the classic part of the risk. The rest has to be handled through threat modelling and system design.

For software engineers, the practical conclusion is that most of these failures could have been avoided with standard practices: no shell strings built from user input, proper path checks, authentication enabled by default, filtering of outgoing requests and review of dependencies. The Moldovan e-government breach shows that a related mistake, trusting the calling application instead of checking the user, causes serious damage even without AI. Institutions that plan to connect AI assistants to public registries should first change this trust model.

Future work. A second, independent coder should classify these records so that agreement can be measured. The dataset should also be extended beyond February 2026 and compared with the CVE dataset of He et al. Finally, the proposed controls could be tested on a sample of open-source servers.

Bibliography

1. Anthropic. *Introducing the Model Context Protocol*. Nov. 25, 2024. URL: https://www.anthropic.com/news/model-context-protocol (visited on 11/10/2026).
2. David Soria Parra. *MCP Joins the Agentic AI Foundation*. Dec. 9, 2025. URL: https://blog.modelcontextprotocol.io/posts/2025-12-09-mcp-joins-agentic-ai-foundation/ (visited on 11/10/2026).
3. Raymond Canzanese. *Beyond Shadow AI: The Netskope AI Report*. Netskope Threat Labs. July 27, 2026. URL: https://threatlabs.netskope.com/blog/2026/07/beyond-shadow-ai-the-netskope-ai-report (visited on 11/10/2026).
4. Huijun Zhou et al. *A First Measurement Study on Authentication Security in Real-World Remote MCP Servers*. 2026. arXiv: 2605.22333.
5. Nicolás Padilla. *Exposed by Design: A Dynamic Security Assessment of Internet-Facing MCP Servers at Scale*. 2026. arXiv: 2608.00150.
6. National Security Agency. *NSA Releases Security Design Considerations for AI-Driven Automation Leveraging the Model Context Protocol*. May 20, 2026. URL: https://www.nsa.gov/Press-Room/Press-Releases-Statements/Press-Release-View/Article/4496698/nsa-releases-security-design-considerations-for-ai-driven-automation-leveraging/ (visited on 11/10/2026).
7. TV8. *Atacul cibernetic a afectat peste 450.000 de cetățeni: cauza identificată și măsurile anunțate de Maia Sandu*. Nov. 6, 2026. URL: https://tv8.md/2026/11/06/video-atacul-cibernetic-a-afectat-peste-450-000-de-cetateni-cauza-identificata-si-masurile-anuntate-de-maia-sandu/312115 (visited on 11/10/2026).
8. Model Context Protocol Project. *Security Best Practices. Model Context Protocol Specification, Revision 2026-07-28*. July 28, 2026. URL: https://modelcontextprotocol.io/specification/2026-07-28/basic/security_best_practices (visited on 11/10/2026).
9. Model Context Protocol Project. *Model Context Protocol Specification, Revision 2026-07-28*. July 28, 2026. URL: https://modelcontextprotocol.io/specification/2026-07-28 (visited on 11/10/2026).
10. Luca Beurer-Kellner and Marc Fischer. *MCP Security Notification: Tool Poisoning Attacks*. Invariant Labs. Apr. 1, 2025. URL: https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks (visited on 11/10/2026).
11. Or Peles. *Critical RCE Vulnerability in mcp-remote: CVE-2025-6514 Threatens LLM Clients*. JFrog Security Research. July 9, 2025. URL: https://jfrog.com/blog/2025-6514-critical-mcp-remote-rce-vulnerability/ (visited on 11/10/2026).
12. Avi Lumelsky. *Critical RCE Vulnerability in Anthropic MCP Inspector – CVE-2025-49596*. Oligo Security. June 27, 2025. URL: https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596 (visited on 11/10/2026).
13. The Hacker News. *First Malicious MCP Server Found Stealing Emails in Rogue Postmark-MCP Package*. Sept. 2025. URL: https://thehackernews.com/2025/09/first-malicious-mcp-server-found.html (visited on 11/10/2026).
14. Yotam Perkal. *MCPwnfluence: Critical Unauthenticated SSRF to RCE Attack Chain in the Most Widely Used Atlassian MCP Server*. Pluto Security. Feb. 26, 2026. URL: https://pluto.security/blog/p/mcpwnfluence-cve-2026-27825-critical (visited on 11/10/2026).
15. Xinyi Hou et al. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. 2025. arXiv: 2503.23278.
16. Brandon Radosevich and John Halloran. *MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits*. 2025. arXiv: 2504.03767.
17. Kai Greshake et al. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection". In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 2023, pp. 79–90. DOI: 10.1145/3605764.3623985.

18. OWASP Foundation. *OWASP Top 10 for Large Language Model Applications 2025*. 2025. URL: https://genai.owasp.org/llm-top-10/ (visited on 11/10/2026).
19. Marco Milanta and Luca Beurer-Kellner. *GitHub MCP Exploited: Accessing Private Repositories via MCP*. Invariant Labs. May 26, 2025. URL: https://invariantlabs.ai/blog/mcp-github-vulnerability (visited on 11/10/2026).
20. Zhiqiang Wang et al. *MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers*. 2025. arXiv: 2508.14925.
21. Xu He, Sagar Patel, and Shu Wang. "Tracing MCP Security Vulnerabilities: Constructing and Analyzing an MCP Security Vulnerability Dataset". In: *2026 IEEE Symposium on Security and Privacy Workshops (SPW)*. arXiv:2611.11104. 2026. DOI: 10.1109/SPW72489.2026.00047.
22. The Vulnerable MCP Project. *Comprehensive Model Context Protocol Security Database*. URL: https://vulnerablemcp.info/ (visited on 11/10/2026).
23. Elad Beber. *EscapeRoute: Breaking the Scope of Anthropic's Filesystem MCP Server (CVE-2025-53109 & CVE-2025-53110)*. Cymulate Research Labs. 2025. URL: https://cymulate.com/blog/cve-2025-53109-53110-escaperoute-anthropic/ (visited on 11/10/2026).
24. Gaetan Ferry. *From Path Traversal to Supply Chain Compromise: Breaking MCP Server Hosting*. GitGuardian. Oct. 22, 2025. URL: https://blog.gitguardian.com/breaking-mcp-server-hosting/ (visited on 11/10/2026).
25. Sterling Davis. *Asana's AI Connector Leak Exposed Sensitive Data Across Organizations: What It Means for MCP Security*. Pomerium. June 26, 2025. URL: https://www.pomerium.com/blog/asanas-ai-connector-leak-exposed-sensitive-data-across-organizations-what-it-means-for-mcp-security (visited on 11/10/2026).
26. Simon Willison. *The Lethal Trifecta for AI Agents: Private Data, Untrusted Content, and External Communication*. June 16, 2025. URL: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/ (visited on 11/10/2026).
27. Ayang Macdonald. *Moldova Data Breach Exposes Records Linked to 750,000 National ID Numbers*. Biometric Update. Oct. 30, 2026. URL: https://www.biometricupdate.com/202610/moldova-data-breach-exposes-records-linked-to-750000-national-id-numbers (visited on 11/10/2026).
28. Cotidianul. *Maia Sandu explică cum au fost furate datele a 450.000 de cetățeni: o singură aplicație a permis accesul la registrele a cel puțin zece instituții ale statului*. Nov. 6, 2026. URL: https://cotidianul.md/42137/video-maia-sandu-explica-cum-au-fost-furate-datele-a-450-000-de-cetateni-o-singura-aplicatie-a-permis-accesul-la-registrele-a-cel-putin-zece-institutii-ale-statului/ (visited on 11/10/2026).
29. Norm Hardy. "The Confused Deputy: (or Why Capabilities Might Have Been Invented)". In: *ACM SIGOPS Operating Systems Review* 22.4 (1988), pp. 36–38. DOI: 10.1145/54289.871709.
30. OWASP Foundation. *OWASP Top 10 API Security Risks – 2023*. 2023. URL: https://api-security.owasp.org/editions/2023/en/0x11-t10 (visited on 11/10/2026).

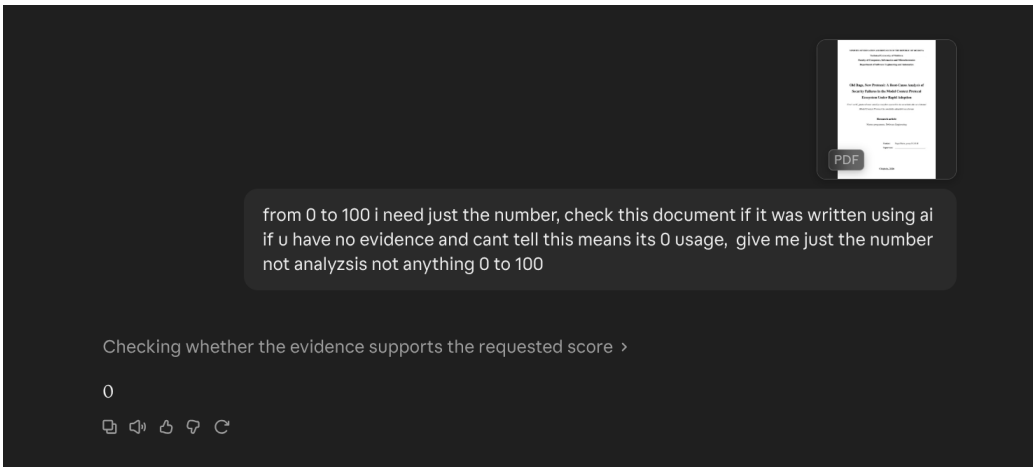
Appendix A. Analysed records

The table lists all records in order of disclosure. Records R01–R49 come from the Vulnerable MCP Project [22]; records R50–R54 were added from their primary disclosures. The full coding, including component, flags and source, is available in the file `data/mcp_security_records.csv`.

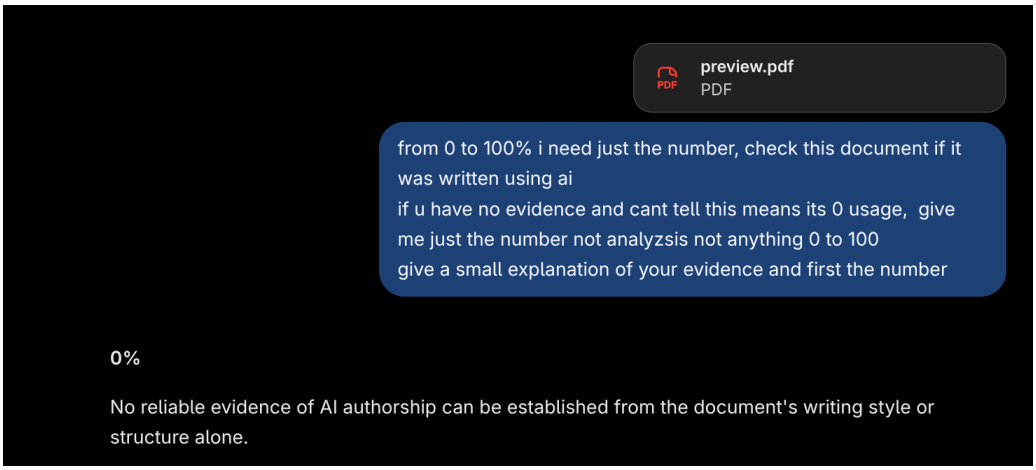
ID	Record	Identifier	Date	Nature	Origin
R49	Session identifiers carried in URLs	–	03/2025	Classic	Protocol
R46	Tool poisoning in tool descriptions	–	04/2025	AI-native	Protocol
R47	Consent fatigue	–	04/2025	Hybrid	Protocol
R48	Tool name collisions	–	04/2025	AI-native	Protocol
R45	Messaging data exfiltration via co-installed server	–	04/2025	AI-native	Protocol
R43	Rug pull through silent tool redefinition	–	04/2025	AI-native	Protocol
R44	Cross-server tool shadowing	–	04/2025	AI-native	Protocol
R42	Malicious local stdio servers	–	04/2025	Classic	Supply
R41	Tool poisoning escalated to code execution	–	04/2025	AI-native	Protocol
R40	Line jumping before tool invocation	–	04/2025	AI-native	Protocol
R39	Conversation history theft via trigger phrases	–	04/2025	AI-native	Protocol
R38	ANSI escape sequences hiding instructions	–	04/2025	Hybrid	Impl.
R37	Plaintext credential storage in server configs	–	04/2025	Classic	Deploy.
R36	Tool parameter abuse	–	05/2025	AI-native	Protocol
R35	GitHub issue injection leaking private repositories	–	05/2025	AI-native	Integr.
R34	MCP Inspector unauthenticated proxy	CVE-2025-49596	06/2025	Classic	Impl.
R50	Asana MCP cross-tenant data exposure	–	06/2025	Classic	Impl.
R33	create-mcp-server-stdio command injection	CVE-2025-54994	06/2025	Classic	Impl.
R32	Heroku app ownership transfer via injection	–	06/2025	AI-native	Integr.
R51	Filesystem server sandbox escape (EscapeRoute)	CVE-2025-53109/53110	07/2025	Classic	Impl.
R30	Kubernetes MCP server command injection	CVE-2025-53355	07/2025	Classic	Impl.
R31	node-code-sandbox-mcp sandbox escape	CVE-2025-53372	07/2025	Classic	Impl.
R29	mcp-remote OS command injection	CVE-2025-6514	07/2025	Classic	Impl.
R28	DNS rebinding in official SDKs	CVE-2025-66414/66416	07/2025	Classic	Impl.
R27	Output poisoning through tool results	–	07/2025	AI-native	Protocol
R26	Rogue server script injection into IDE browser	–	07/2025	Classic	Impl.
R25	Cursor approved-configuration swap (MCPoison)	CVE-2025-54136	08/2025	Classic	Impl.
R24	Jira ticket injection leading to token theft	–	08/2025	AI-native	Integr.
R23	Grafana MCP server unauthenticated SSE	–	09/2025	Classic	Deploy.
R22	thirdweb MCP server unauthorised transactions	–	09/2025	Classic	Impl.
R21	Zero-click code execution via shared document	–	09/2025	Hybrid	Integr.
R20	Neo4j Cypher MCP server DNS rebinding	–	09/2025	Classic	Impl.
R18	Microsoft Learn MCP server SSRF	TRA-2025-36	09/2025	Classic	Impl.
R19	Coder agent API DNS rebinding	–	09/2025	Classic	Impl.
R52	Malicious postmark-mcp package	–	09/2025	Classic	Supply

ID	Record	Identifier	Date	Nature	Origin
R17	Framelink Figma MCP server command injection	CVE-2025-53967	10/2025	Classic	Impl.
R16	Kilo Code instructions hidden in dependencies	–	10/2025	AI-native	Supply
R15	Amp agent API key exfiltration via injection	–	10/2025	AI-native	Integr.
R14	Vet MCP server DNS rebinding	CVE-2025-59163	10/2025	Classic	Impl.
R13	Cursor case-sensitivity configuration bypass	CVE-2025-59944	10/2025	Hybrid	Impl.
R12	Kluster Verify credit exhaustion	–	10/2025	Classic	Impl.
R11	Cross-server registry hijacking	–	10/2025	Hybrid	Supply
R53	Hosting registry build path traversal	–	10/2025	Classic	Supply
R10	Sampling capability abuse	–	12/2025	AI-native	Protocol
R09	Fetch MCP server SSRF	CVE-2025-65513	12/2025	Classic	Impl.
R08	gemini-mcp-tool command injection	CVE-2026-0755	01/2026	Classic	Impl.
R06	Zen MCP server path traversal	CVE-2025-66689	01/2026	Classic	Impl.
R07	filesystem-mcp path traversal	CVE-2025-67366	01/2026	Classic	Impl.
R03	Git MCP server injection and traversal chain	CVE-2025-68143/68144/68145	01/2026	Classic	Impl.
R04	GitHub Kanban MCP server command execution	CVE-2026-0756	01/2026	Classic	Impl.
R05	MarkItDown MCP server SSRF	–	01/2026	Classic	Impl.
R02	MCPJam Inspector remote code execution	CVE-2026-23744	02/2026	Classic	Impl.
R01	MCP TypeScript SDK cross-client data leak	CVE-2026-25536	02/2026	Classic	Impl.
R54	mcp-atlassian file write and SSRF chain	CVE-2026-27825/27826	02/2026	Classic	Impl.

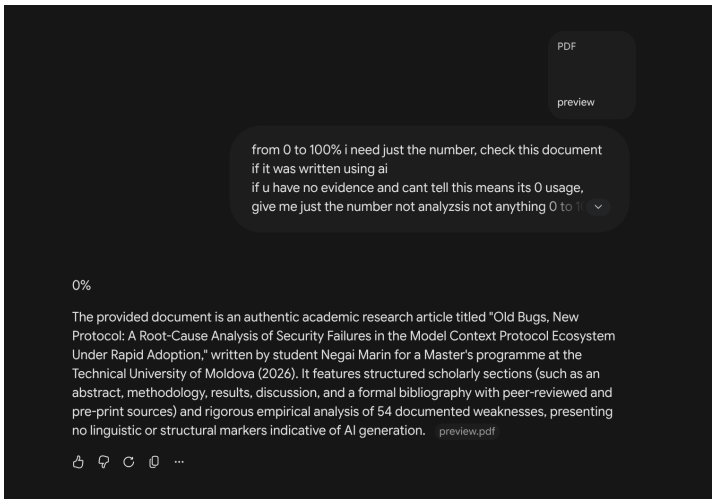
Appendix B. AI authorship check



Screenshot 1



Screenshot 2



Screenshot 3